

The Expectation of Incompleteness

A Formal Account of Knowledge Graphs as Instruments That Report What They Cannot Say

Kundai Farai Sachikonye

Experimental Bioinformatics, TUM School of Life Sciences

Maximus-von-Imhof-Forum 3, 85354 Freising, Germany

kundai.sachikonye@bitspark.com

August 16, 2026

Abstract

A causal knowledge graph over any biochemical corpus will fail to account for some of the reactions in that corpus. This is not a defect to be engineered away; it is the expected state of every such artefact, and the interesting question is not how much is covered but whether the system can say, at each question it is asked, *which* of its several independent limitations is responsible for the failure. We argue that a knowledge graph should be built as a measuring instrument whose primary output is a verdict rather than an answer set, and we develop the formal apparatus for this from first principles. We define three logically independent predicates over a question—*expressible* in a model, *derivable* from a corpus under that model’s semantics, and *answerable* by a given engine—and prove that no two of them entail the third, so that the customary practice of reporting a single empty answer set conflates three distinct claims. We introduce a verdict space in which CANNOT-EXPRESS, NOT-DERIVABLE and NO-QUERY-SURFACE are outcomes with the same evidential standing as ANSWERED, and prove a *non-degeneracy* property: no verdict other than ANSWERED may carry an empty answer set, and ANSWERED may not carry one either unless emptiness is itself certified. We then show that a system’s declared capability set is a claim about its *compiler* and not about its underlying formalism, that the two claims come apart in both directions, and that only the compiler-level reading admits a decidable containment check. Because a constraint may be lowered into a target language in a form that is syntactically present and semantically inert, we prove that syntactic and semantic vacuity checks are mutually irredundant—neither subsumes the other—and that a system carrying only one of them can certify a result produced by a constraint that did no work. Finally we prove a *single-derivation* property: when every view of a result is computed from one derivation, two views of the same system cannot disagree, and we show that duplicated derivation admits contradictory publication with every schema check passing. Seventeen computational experiments accompany the paper. Each is stated as a proposition, given a control able to fail, and reported with its verdict; several are designed so that the interesting outcome is a refusal. Everything required to read this paper is developed here from first principles, with no reliance on prior work by the author.

Contents

1	Introduction	2
1.1	The failure mode that motivates this paper	2
1.2	Why incompleteness is the expected state	3
1.3	What follows from taking incompleteness seriously	3
1.4	Relation to existing work, and what is new	4
1.5	Organisation	4
2	Setting	4

3	Three independent predicates	5
3.1	Definitions	5
3.2	Independence	6
3.3	The specific structure of counting	7
4	The verdict space	7
4.1	Construction	7
4.2	Non-degeneracy	9
4.3	Refusal as a measurement	9
5	Capability is a property of the compiler	10
5.1	The containment check	10
5.2	The formalism can do what the compiler cannot	10
5.3	One name for two mechanisms	11
5.4	Nameability	11
6	Vacuity	11
7	The blocker trichotomy	12
8	Divergent semantics as a result	13
8.1	Bounding a question can make it cheaper	14
9	One derivation	15
10	Computational validation	16
10.1	Method	16
10.2	The propositions under test	17
10.3	Results	17
10.4	What the suite does not establish	18
11	Discussion	18
11.1	What the framework buys	18
11.2	What the framework does not do	18
11.3	Outlook	19

1 Introduction

1.1 The failure mode that motivates this paper

Consider a system that answers questions over a biochemical corpus by compiling each question into some logical form, running a reasoner, and returning the resulting set of entities. Ask it: *which reactions have exactly four participants?* Suppose the corpus contains three such reactions and the system returns the empty set.

There are at least five distinct situations in which that output is produced. The model may lack cardinality constructs altogether, so the question was never representable. The model may have them, but under an open-world reading the count is not entailed, because nothing rules out a fourth participant that merely was not recorded. The corpus may genuinely contain no such reaction. The compiler may have silently dropped the cardinality restriction while lowering the question, so that a syntactically well-formed query ran and matched nothing. Or the reasoner may have exhausted a search budget and returned partial results without saying so.

These five situations warrant five different actions on the part of whoever receives the answer, and exactly one output is produced for all of them. The output is not wrong in any way that

inspection reveals: it is well formed, it is of the correct type, it is returned promptly, and in the second case it is even *correct* under the semantics the system nominally implements. It is simply uninformative in a way that is invisible.

This paper is about building systems in which those five situations produce five different outputs, and about what that requirement does to the architecture. Our thesis is that the requirement is not a reporting nicety bolted onto a query engine but a structural constraint that determines what the system is: a knowledge graph built this way is an *instrument*, and its output at each question is a *verdict*, of which an answer set is one possible constituent.

1.2 Why incompleteness is the expected state

It is worth being explicit that we do not regard the incompleteness of a biochemical knowledge graph as a temporary condition. Three independent sources guarantee it.

The first is expressive. Any model is a fragment of some logic, and every decidable fragment of first-order logic omits constructions that some question will need [Baader et al., 2017]. The trade-off between expressive power and the tractability of reasoning is not an engineering artefact but a theorem-level fact about the fragments [Donini, 2003, Grau et al., 2008]. A model chosen so that reasoning terminates is a model in which some questions are not statable.

The second is empirical. Curated biochemical corpora are assembled by human effort under finite budget from an open and growing literature. Reaction databases record what has been curated, not what exists [Alcántara et al., 2012, Bansal et al., 2022], and chemical ontologies partition molecules along axes chosen for chemical rather than metabolic convenience [Degtyarenko et al., 2008, Hastings et al., 2016], so that species a biochemist would group together may share no informative common ancestor. Absence in a corpus is not evidence of absence in the world, and treating it as such is the closed-world error.

The third is computational. Even where a question is both statable and entailed, the reasoner may not finish. Description-logic reasoning is intractable in the worst case for the expressive profiles [Motik et al., 2009b, Glimm et al., 2014], and the profiles designed for tractability [Motik et al., 2009a, Kazakov et al., 2014] achieve it precisely by dropping constructs. A question that cannot be run on the available machine within the available time is unanswered in a way that has nothing to do with the first two.

These three sources are independent. A question may be inexpressible in a model whose corpus is rich and whose engine is fast; it may be expressible and supported but unentailed; it may be expressible, entailed and beyond the budget. Our formal development takes this independence as its starting point, and Theorem 3.4 makes it precise.

1.3 What follows from taking incompleteness seriously

If incompleteness is expected, then the informative content of a knowledge graph is not only the set of questions it answers. It is the *structured account of the questions it does not answer*. A system that answers sixty per cent of a question set and cannot characterise the other forty is less useful than one that answers forty per cent and, for each of the remaining sixty, states which of the three sources is responsible and what would have to change to unblock it. The second system supports a decision; the first supports only a percentage.

This reframing has four consequences that we develop in the body of the paper.

- (C1) **Refusal is a first-class outcome.** A verdict of CANNOT-EXPRESS is a result. It is derived, it is reproducible, it constrains future work, and it may be cited. Treating refusal as a failure to be minimised makes an honest refusal look worse than a confident wrong answer. (Section 4.)
- (C2) **A capability claim describes a compiler, not a formalism.** The statement “this system supports counting” is true or false of a lowering procedure, not of the target logic.

The two come apart in both directions, and only the compiler-level reading supports a decidable containment check. (Section 5.)

- (C3) **Every constraint carries a control that can invalidate its own result.** A constraint may be present in the compiled form and yet do no work. Detecting this requires two checks that do not subsume one another. (Section 6.)
- (C4) **Derivation happens once.** Every view of a result is computed from a single derivation, so that two presentations of the same system cannot disagree. (Section 9.)

1.4 Relation to existing work, and what is new

Open-world semantics and the absence of a unique-name assumption are standard in description logic [Baader et al., 2017, Horrocks et al., 2003], and the distinction between an empty extension and an untailed count is not new. Benchmarking suites for ontology reasoning report per-engine timeouts and errors [Guo et al., 2005, Singh et al., 2020, Parsia et al., 2017], and it is common practice to report a reasoner’s failure to terminate rather than to record a zero.

What is new here is threefold. First, the separation of *inexpressible in the logic* from *unaskable through this interface*: benchmark suites typically report an engine failure, folding together a limitation of a profile with a limitation of a particular adapter, and the fold systematically converts an implementation defect into a claim about a standard. Second, the treatment of *vacuity* as a first-class hazard requiring two irredundant checks, which to our knowledge has not been formalised for constraint-lowering pipelines even though the analogous notion of vacuous satisfaction is well developed in model checking [Beer et al., 2001, Kupferman and Vardi, 2003]. Third, the *non-degeneracy* requirement—no verdict other than ANSWERED may carry an empty answer set— which turns the conflation described in §1.1 into a checkable structural property of a system rather than a matter of reporting discipline.

1.5 Organisation

Section 2 fixes the setting and defines models, corpora, engines, questions and features. Section 3 defines expressibility, derivability and answerability and proves their pairwise independence. Section 4 constructs the verdict space and proves non-degeneracy. Section 5 develops capability sets as compiler-level claims and proves the containment theorem and the two failure directions. Section 6 treats vacuity and proves the irredundance of the two checks. Section 7 gives the blocker trichotomy and the unblocking relation. Section 8 treats the divergence of semantics between target languages, showing that the same model lowered to two engines yields legitimate disagreement that is a result rather than a defect. Section 9 proves the single-derivation property and the staleness theorem for status fields. Section 10 states the seventeen computational propositions and reports their measured outcomes. Section 11 discusses limits and what the framework does not do.

2 Setting

We work with four kinds of object. Throughout, all sets are finite unless stated otherwise.

Definition 2.1 (Signature). A *signature* $\Sigma = (N_C, N_R, N_I)$ consists of disjoint finite sets of *concept names*, *role names* and *individual names*.

Definition 2.2 (Model). A *model* $\mathcal{M} = (\Sigma, \mathcal{T})$ is a signature together with a finite set $\mathcal{T}$ of axioms over Σ , drawn from a fixed axiom grammar. We write $\mathcal{F}(\mathcal{M}) \subseteq \mathcal{F}$ for the set of *features* occurring in $\mathcal{T}$, where $\mathcal{F}$ is a fixed finite alphabet of construct names fixed in Theorem 2.5.

Definition 2.3 (Corpus). A *corpus* $\mathcal{D}$ over Σ is a finite set of ground assertions $A(a)$ and $r(a, b)$ with $A \in N_C$, $r \in N_R$, $a, b \in N_I$. The pair $(\mathcal{M}, \mathcal{D})$ is a *knowledge base*.

We deliberately separate $\mathcal{M}$ from $\mathcal{D}$. The separation is what makes the second and third blockers of Section 7 distinguishable: a question may be unanswerable because the axioms cannot state it or because the assertions do not populate it, and merging the two into a single “ontology” makes those cases indistinguishable by construction.

Definition 2.4 (Question). A *question* q is a pair $(\varphi, \bar{x})$ where φ is a formula over Σ in a fixed query grammar and $\bar{x}$ is a tuple of its free variables. We write $\mathcal{Q}$ for the set of questions. Each question carries a *feature requirement* $\varrho(q) \subseteq \mathcal{F}$: the set of constructs that any faithful rendering of q must use.

Definition 2.5 (Feature alphabet). $\mathcal{F}$ is the finite alphabet

$$\mathcal{F} = \{ \text{SUBSUMPTION, EXISTENTIAL, VALUE, NEGATION, COUNTING, TRANSITIVE_AXIOM, TRANSITIVE_QUERY, DISJOINTNESS, UNIQUE_NAMES} \}.$$

Two remarks on this alphabet are load-bearing and are proved later.

Remark 2.6 (Splitting one name into two). Transitivity appears twice. TRANSITIVE_AXIOM is the assertion that a role r is transitive, discharged by the reasoner when it closes the interpretation. TRANSITIVE_QUERY is the requirement that a query compute the transitive closure of a role at answering time. These are distinct mechanisms: a system may implement either without the other. Naming them identically makes a containment check unsound in a way Theorem 5.7 makes precise.

Remark 2.7 (Listing what you do lower). DISJOINTNESS and UNIQUE_NAMES are features of a *model*, not of a question, and it is tempting to omit them from an alphabet indexed by questions. Omitting them is what makes a defect undetectable: if the alphabet cannot name a construct, then a compiler that silently ignores that construct in $\mathcal{T}$ produces a knowledge base differing from its sibling’s, and any comparison between the two engines attributes the difference to the engines. Theorem 5.9 states this.

Definition 2.8 (Engine). An *engine* $\mathcal{E} = (\Lambda_{\mathcal{E}}, \text{Sup}(\mathcal{E}), \text{solve}_{\mathcal{E}})$ consists of a partial *lowering* map $\Lambda_{\mathcal{E}}$ taking a model and a question to a program in the engine’s target language, a *declared capability set* $\text{Sup}(\mathcal{E}) \subseteq \mathcal{F}$, and a *solver* $\text{solve}_{\mathcal{E}}$ taking a program and a corpus to an answer set together with a certification flag and an elapsed time.

Assumption 2.9 (Determinism and budget). $\text{solve}_{\mathcal{E}}$ is deterministic given $(\text{program}, \mathcal{D})$ and is run under an explicit wall-clock budget τ . Any run exceeding τ is terminated and produces no answer set.

Theorem 2.9 is not a technicality. It is what makes TIMEOUT a verdict rather than a crash: a terminated run that returned an empty answer set would be recorded by any differential comparison as *agreement* between a working engine and a hung one.

3 Three independent predicates

3.1 Definitions

Definition 3.1 (Expressible). A question q is *expressible in* $\mathcal{M}$, written $\text{Exp}(q, \mathcal{M})$, if the constructs $\varrho(q)$ are all available in the axiom and query grammar over $\Sigma(\mathcal{M})$, and every name occurring in q is in $\Sigma(\mathcal{M})$.

Expressibility is a property of the *language*. It does not depend on the corpus, and it does not depend on any engine.

Definition 3.2 (Derivable). Let $q = (\varphi, \bar{x})$ be expressible in $\mathcal{M}$. A tuple $\bar{a}$ of individuals is a *certain answer* if $(\mathcal{M}, \mathcal{D}) \models \varphi[\bar{a}/\bar{x}]$, i.e. $\varphi[\bar{a}]$ holds in every model of the knowledge base. The question is *derivable*, written $\text{Der}(q, \mathcal{M}, \mathcal{D})$, if the answer to q is determined by the knowledge base: for every tuple $\bar{a}$ over N_I , either $(\mathcal{M}, \mathcal{D}) \models \varphi[\bar{a}]$ or $(\mathcal{M}, \mathcal{D}) \models \neg\varphi[\bar{a}]$.

Derivability is a property of the *semantics*. Note carefully that it is stronger than the existence of certain answers: a question with no certain answers may still be underdetermined, and the difference between “the extension is empty” and “the extension is not determined” is exactly the one §1.1 identified as invisible.

Definition 3.3 (Answerable). A question q is *answerable by $\mathcal{E}$ on $(\mathcal{M}, \mathcal{D})$ within τ* , written $\text{Ans}_\tau(q, \mathcal{M}, \mathcal{D}, \mathcal{E})$, if $\Lambda_{\mathcal{E}}(\mathcal{M}, q)$ is defined and $\text{solve}_{\mathcal{E}}$ on the resulting program and $\mathcal{D}$ terminates within τ with certification.

Answerability is a property of the *implementation and the budget*. It mentions a clock. Neither of the other two does.

3.2 Independence

Theorem 3.4 (Pairwise independence). *None of Exp , Der , Ans_τ is entailed by the conjunction of the other two. Formally, for each of the three predicates there exist a model, a corpus, an engine, a budget and a question witnessing the other two and failing that one.*

Proof. We exhibit three witnesses.

Der and Ans_τ without Exp . Let $\mathcal{M}_1$ have grammar without cardinality restrictions, so $\text{COUNTING} \notin \mathcal{F}(\mathcal{M}_1)$ and no question requiring it is expressible. Take $q_1 =$ “entities with exactly four r -successors”. Now consider the surrogate question $q'_1 =$ “entities x with $r(x, a_1), \dots, r(x, a_4)$ for the four named individuals $a_1, \dots, a_4$ ”, which *is* expressible and which, on a corpus asserting exactly those four edges for one individual and no others, is derivable and answerable by any engine handling conjunctive queries. The pair witnesses that expressibility fails on q_1 while the semantic and computational conditions are unproblematic for the surrogate; formally, $\text{Exp}(q_1, \mathcal{M}_1)$ is false while for the extension $\mathcal{M}_1^+$ obtained by adding cardinality to the grammar, $\text{Der}(q_1, \mathcal{M}_1^+, \mathcal{D})$ and $\text{Ans}_\tau(q_1, \mathcal{M}_1^+, \mathcal{D}, \mathcal{E})$ both hold. Since $\mathcal{D}$ and $\mathcal{E}$ are unchanged, the failure is attributable to the grammar alone.

Exp and Ans_τ without Der . Let $\mathcal{M}_2$ have cardinality restrictions, and let $\mathcal{D}_2$ assert $r(c, a_1), \dots, r(c, a_4)$ with no disjointness among the a_i and no closure axiom on r . Then q_1 is expressible, and an engine returns promptly. But $(\mathcal{M}_2, \mathcal{D}_2) \not\models$ “ c has exactly four r -successors”, because there is a model of the knowledge base in which $a_1 = a_2$ (no unique-name assumption) and another in which c has a fifth successor (no closure). Nor does the knowledge base entail the negation, since a model realising exactly four distinct successors exists. So the question is underdetermined: Der fails while the other two hold.

Exp and Der without Ans_τ . Let $\mathcal{M}_3$ and $\mathcal{D}_3$ be such that q is expressible and determined—for instance, take $\mathcal{D}_3$ to assert pairwise distinctness of the a_i and a closure axiom on r at c , which fixes the count—and let τ be smaller than the time $\text{solve}_{\mathcal{E}}$ requires on $\Lambda_{\mathcal{E}}(\mathcal{M}_3, q)$. Such a τ exists because the solver’s running time on a fixed input is a positive real. Then Ans_τ fails while the other two hold. Alternatively, and without appeal to the budget, let $\Lambda_{\mathcal{E}}$ be undefined on q —the engine’s adapter has no surface for cardinality—which also falsifies Ans_τ while leaving Exp and Der untouched. $\square$

Corollary 3.5 (The conflation). *A system whose output on a question is a single answer set cannot distinguish $\neg\text{Exp}$, $\neg\text{Der}$ and $\neg\text{Ans}_\tau$, since all three admit the empty set as output and, by Theorem 3.4, no two of them determine the third.*

Remark 3.6 (The two witnesses of $\neg\text{Ans}_\tau$ are different). The proof gave two independent ways to falsify answerability: the budget and the undefined lowering. They warrant different verdicts and

different actions, and Section 4 separates them as TIMEOUT and NO-QUERY-SURFACE. Merging them is how “this adapter has no cardinality surface” becomes “this profile cannot count”.

3.3 The specific structure of counting

The second witness in Theorem 3.4 deserves a sharper statement, because it is the case most often mishandled.

Proposition 3.7 (Two requirements, one aggregate). *Let $q_{\geq n}$, $q_{\leq n}$ and $q_{=n}$ be the questions “ x has at least / at most / exactly n r -successors”. Under open-world semantics without a unique-name assumption:*

- (i) $q_{\geq n}$ is derivable only if the counted successors are known pairwise distinct;
- (ii) $q_{\leq n}$ is derivable only if the successor set is closed;
- (iii) $q_{=n}$ requires both.

Proof. (i) Suppose $r(c, a_1), \dots, r(c, a_n)$ hold and distinctness is not asserted. The interpretation identifying all a_i satisfies the knowledge base and gives c one successor, so “at least n ” is not entailed for $n > 1$. Conversely if the a_i are pairwise distinct in every model then every model gives c at least n successors. (ii) Suppose the successor set is not closed. Extend any model by a fresh element d with $r(c, d)$; if the extension satisfies the knowledge base, “at most n ” fails in it. Conversely a closure axiom $c \sqsubseteq \forall r.\{a_1, \dots, a_n\}$ makes every model’s successor set a subset of $\{a_1^{\mathcal{I}}, \dots, a_n^{\mathcal{I}}\}$, bounding the count by n . (iii) $q_{=n}$ is the conjunction, and a conjunction is entailed only if both conjuncts are. $\square$

Remark 3.8 (The tempting check is wrong in the hiding direction). A system implementing Theorem 3.7(i) must check that the *counted individuals* are known distinct. It is tempting instead to check whether the model contains *any* distinctness assertion, because that is a single boolean over $\mathcal{T}$. This is unsound in the direction that conceals the problem: a model may assert distinctness over an unrelated pair of individuals—two role constants, say—while saying nothing about the successors being counted, and the boolean is then true while the requirement of (i) is unmet. A distinctness assertion somewhere in a model is not a distinctness assertion over the counted set.

Remark 3.9 (False by construction, not by inspection). If a model’s grammar contains no closure construct then Theorem 3.7(ii) fails for structural reasons, and a system should record this fact as *holding by construction*. The distinction matters for maintenance: a fact recorded as “false by construction” is flipped by extending the grammar, whereas a fact recorded as “false by inspection” invites someone to look for a closure axiom that the grammar cannot express. In the first framing, extending the grammar is what changes the verdict; in the second, a test passing is.

4 The verdict space

4.1 Construction

Definition 4.1 (Verdict). A *verdict* is a pair $V = (\ell, \pi)$ where ℓ is a label from

$L = \{\text{ANSWERED, CANNOT-EXPRESS, NOT-DERIVABLE, NO-QUERY-SURFACE, TIMEOUT, CONTROL-VACUOUS, C}$

and π is a label-specific *payload*. The payloads are:

Label	Payload
ANSWERED	answer set A , certification flag, elapsed time
CANNOT-EXPRESS	the subset $\varrho(q) \setminus \mathcal{F}(\mathcal{M})$ responsible
NOT-DERIVABLE	which clause of Theorem 3.7 (or analogue) failed
NO-QUERY-SURFACE	the feature the lowering has no surface for
TIMEOUT	the budget τ that was exceeded
CONTROL-VACUOUS	the compiled expressions found identical
CONSTRAINT-INERT	the two answer sets found identical

Definition 4.2 (Evaluation). Given $(\mathcal{M}, \mathcal{D}, \mathcal{E}, \tau)$ and a question q , the *evaluation* $\text{Ev}(q)$ is computed in the following order, the first applicable clause determining the verdict:

- (E1) if $\varrho(q) \not\subseteq \mathcal{F}(\mathcal{M})$, return (CANNOT-EXPRESS, $\varrho(q) \setminus \mathcal{F}(\mathcal{M})$);
- (E2) if q is not derivable by the applicable clause of Theorem 3.7, return (NOT-DERIVABLE, clause);
- (E3) if $\varrho(q) \not\subseteq \text{Sup}(\mathcal{E})$, return (NO-QUERY-SURFACE, $\varrho(q) \setminus \text{Sup}(\mathcal{E})$);
- (E4) compile $P = \Lambda_{\mathcal{E}}(\mathcal{M}, q)$ and, if q carries a droppable constraint, the control P° obtained by removing it. If $P = P^\circ$ syntactically, return (CONTROL-VACUOUS, (P, P°));
- (E5) run P within τ ; if it exceeds τ , return (TIMEOUT, τ). Run P° within τ ; if the control alone exceeds it, the vacuity test of (E6) is *inconclusive* and is recorded as such in the payload of whatever clause below applies;
- (E6) if $A(P) = A(P^\circ)$ and $A(P) \neq \emptyset$, return (CONSTRAINT-INERT, $A(P)$);
- (E7) otherwise return (ANSWERED, $(A(P), \text{certified}, t)$).

The order is not arbitrary. Each clause is checked before any clause whose evidence it would corrupt. In particular (E1) precedes (E3), so that a limitation of the language is never reported as a limitation of the adapter; and (E4) precedes (E5), so that a vacuous control is detected before a budget is spent proving that it agrees.

Two side conditions in (E5) and (E6) deserve comment, because the natural statement of each is wrong in a way that only shows up when the clause fires.

Remark 4.3 (The control’s budget is not the question’s). A control that exceeds τ is not a TIMEOUT verdict for q . The question has been answered; what has failed is the auxiliary test of whether its constraint did any work. Returning TIMEOUT here would report a property of P° as a verdict on q — precisely the conflation of Theorem 3.5, one level down. The case is not exotic: whenever the droppable constraint is the one that made the question affordable, the control is by construction the more expensive program, so this is the *expected* behaviour for exactly the questions of Theorem 8.6 rather than a rare degeneracy. An ANSWERED verdict whose vacuity test did not complete must carry that fact in its payload, since otherwise ANSWERED silently implies a check that was never performed.

Remark 4.4 (Two empty sets agree about nothing). The non-emptiness side condition in (E6) is what prevents a capability defect from being reported as a vacuity finding. Theorem 6.3 infers that a constraint did no work from the observation that two *differing* programs produce the same answers, and that inference requires the agreement to have content. If neither program grounds — the ungrounded-aggregate case of Theorem 5.4 — both return $\emptyset$ and agree vacuously, for a reason having nothing to do with the constraint. Without the side condition, a compiler emitting a program that never runs is labelled CONSTRAINT-INERT, which is a verdict about the question rather than about the compiler, and the defect is concealed by the very machinery built to expose it.

Proposition 4.5 (Order soundness). *If $\text{Ev}(q) = (\ell, \pi)$ with $\ell \neq \text{ANSWERED}$ then every clause preceding ℓ 's clause in Theorem 4.2 is inapplicable, and the payload π names a witness sufficient to reproduce the verdict without re-running the engine.*

Proof. Immediate from the first-applicable rule. For the second part, inspect the payload column of Theorem 4.1: each payload is a finite object computed from $\mathcal{M}$, q , $\text{Sup}(\mathcal{E})$, $\Lambda_{\mathcal{E}}$ and τ alone, none of which depends on the solver's execution except in the **ANSWERED** and **CONSTRAINT-INERT** rows, whose payloads include the answer sets themselves. $\square$

4.2 Non-degeneracy

The property that makes the verdict space more than a taxonomy is the following.

Theorem 4.6 (Non-degeneracy). *Under Theorem 4.2, no verdict with $\ell \neq \text{ANSWERED}$ carries an empty answer set, because no such verdict carries an answer set at all. Consequently the observation “the output was empty” determines $\ell = \text{ANSWERED}$, and the five situations of §1.1 are pairwise distinguishable from the verdict alone.*

Proof. By inspection of the payload table: the only labels whose payload contains an answer set are **ANSWERED** and **CONSTRAINT-INERT**. **CONSTRAINT-INERT** is reached only when $A(P) = A(P^\circ)$, and its payload records that common set as evidence of inertness rather than as an answer, so no consumer reading the label as an answer set is licensed. For the second claim, suppose a consumer observes an empty output. By the first part the label must be **ANSWERED**. The five situations of §1.1 are then separated as follows: an inexpressible question yields **CANNOT-EXPRESS** by (E1); an untailed count yields **NOT-DERIVABLE** by (E2); a genuinely empty extension yields **(ANSWERED, $(\emptyset, \text{certified}, t)$)**; a dropped constraint yields **CONTROL-VACUOUS** by (E4) if the drop is syntactic and **CONSTRAINT-INERT** by (E6) if it is semantic; an exhausted budget yields **TIMEOUT** by (E5). These are five distinct labels. $\square$

Corollary 4.7 (Certified emptiness is a result). *A verdict **(ANSWERED, $(\emptyset, \text{certified}, t)$)** is a positive claim: the extension is empty and the run that established this was complete. It is therefore not interchangeable with any $\ell \neq \text{ANSWERED}$, and in particular not with **TIMEOUT**, whose payload asserts nothing about the extension.*

Remark 4.8 (Certification is reported, never folded in). The certification flag is a component of the **ANSWERED** payload and not a filter on it. A system that dropped uncertified answers would report a search-limited run and a complete run identically when both return the same set, and would report a search-limited run and an exhausted one identically when the former returns nothing. Where a solver mode is constitutionally unable to certify, the flag must be explicitly false rather than absent: an unset field reads as a default and a default reads as satisfactory.

Remark 4.9 (The budget is part of the claim). **(TIMEOUT, 120s)** and **(TIMEOUT, 3600s)** are different claims, and a verdict space that records only the label loses the difference. This is why Theorem 4.1 makes τ the payload rather than metadata.

4.3 Refusal as a measurement

Proposition 4.10 (Comparability of refusals). *Let $\mathcal{E}_1, \mathcal{E}_2$ be engines evaluated on the same $(\mathcal{M}, \mathcal{D}, q, \tau)$. Then the pair $(\text{Ev}_1(q), \text{Ev}_2(q))$ is informative even when neither label is **ANSWERED**, and in particular the pair **(CANNOT-EXPRESS, ANSWERED)** is not evidence that $\mathcal{E}_2$ is more capable than $\mathcal{E}_1$ on q unless $\mathcal{E}_2$'s payload includes certification.*

Proof. Suppose $\text{Ev}_1(q) = (\text{CANNOT-EXPRESS}, \{\text{COUNTING}\})$ and $\text{Ev}_2(q) = (\text{ANSWERED}, (\emptyset, \cdot, t))$ on a corpus whose ground truth for q is a non-empty set of size $m > 0$. Then $\mathcal{E}_1$'s output correctly identifies that q cannot be posed in its language, while $\mathcal{E}_2$'s output differs from ground truth by m . Ranking $\mathcal{E}_2$ above $\mathcal{E}_1$ on the basis that it produced an answer set orders them by output type

rather than by correctness, and reverses the correct ordering whenever $m > 0$. If $\mathcal{E}_2$'s empty set is certified, then by Theorem 4.7 its verdict is a positive claim of emptiness, which contradicts $m > 0$ and identifies a defect; if it is uncertified, the verdict carries no claim about the extension at all. $\square$

Theorem 4.10 is the formal statement of the phenomenon that motivated this paper. An engine that refuses honestly appears *less* capable than one returning a confident wrong zero, under any scoring rule that counts answers rather than verdicts.

5 Capability is a property of the compiler

5.1 The containment check

Definition 5.1 (Faithful lowering). $\Lambda_{\mathcal{E}}$ is *faithful* for $f \in \mathcal{F}$ if for every model and question in which f occurs, the program $\Lambda_{\mathcal{E}}(\mathcal{M}, q)$ has the same certain answers as q has under Theorem 3.2.

Definition 5.2 (Honest capability set). $\text{Sup}(\mathcal{E})$ is *honest* if $f \in \text{Sup}(\mathcal{E})$ iff $\Lambda_{\mathcal{E}}$ is faithful for f .

Theorem 5.3 (Containment). *Let $\text{Sup}(\mathcal{E})$ be honest. If $\varrho(q) \cup \mathcal{F}(\mathcal{M}) \subseteq \text{Sup}(\mathcal{E})$ then $\Lambda_{\mathcal{E}}(\mathcal{M}, q)$ is defined and faithful. If $\varrho(q) \cup \mathcal{F}(\mathcal{M}) \not\subseteq \text{Sup}(\mathcal{E})$ then Theorem 4.2(E3) fires and the verdict is (NO-QUERY-SURFACE, $\cdot$), which by Theorem 4.6 carries no answer set.*

Proof. The first part is Theorem 5.2 applied featurewise, using that faithfulness is closed under the conjunction of independent constructs by Theorem 5.1. The second is by construction of (E3), and the absence of an answer set is Theorem 4.6. $\square$

The theorem is only as strong as its hypothesis, and the hypothesis is what the next two subsections attack.

5.2 The formalism can do what the compiler cannot

Proposition 5.4 (Under-declaration is correct, not modest). *There exist a feature f , an engine $\mathcal{E}$ and a target language T such that T supports f in full generality, $\Lambda_{\mathcal{E}}$ produces a program in T using f , that program is rejected or fails to certify, and honesty of $\text{Sup}(\mathcal{E})$ therefore requires $f \notin \text{Sup}(\mathcal{E})$.*

Proof. Take $f = \text{COUNTING}$ and T a rule language with aggregate support. Counting is expressible in T in full generality. Let $\Lambda_{\mathcal{E}}$ emit, for the question “entities with exactly four r -successors”, a rule whose body pairs an aggregate over an unbound successor variable with an equality test on the aggregate’s result. If the engine certifies answers only within a fragment for which grounding is bounded, this program falls outside that fragment: the aggregate’s grounding is unbounded in the successor variable and the engine rejects it. Removing the fragment gate does not repair the situation—it produces a run that terminates with an empty answer set, which is the answer set of a program whose aggregate never grounded, and which by Theorem 4.7 would be read as a positive claim of emptiness. Since $\Lambda_{\mathcal{E}}$ is faithful for f in neither configuration, Theorem 5.2 forces $f \notin \text{Sup}(\mathcal{E})$, notwithstanding that T counts perfectly well. $\square$

Corollary 5.5 (A capability recommendation must be tested by attempting it). *An audit of $\text{Sup}(\mathcal{E})$ conducted against the target language T rather than against $\Lambda_{\mathcal{E}}$ will recommend adding features that must not be added. Such a recommendation is refuted only by writing the lowering and observing its verdict.*

Remark 5.6 (One set cannot carry two positions). $\text{Sup}(\mathcal{E})$ as defined is a single set checked against both $\mathcal{F}(\mathcal{M})$ and $\varrho(q)$. It therefore cannot express “supported when the construct occurs in a query, unsupported when it occurs in an axiom”, which is a real distinction: lowering a construct

into a rule body and lowering it into a rule head are different problems with different fragment conditions. A system needing that distinction requires two sets, and a system with one set must record which position its declaration refers to.

5.3 One name for two mechanisms

Proposition 5.7 (Half-true containment). *Suppose $\mathcal{F}$ contains a single symbol **TRANSITIVE** conflating Theorem 2.6’s two mechanisms. Then there exists an engine $\mathcal{E}$ for which **TRANSITIVE** $\in \text{Sup}(\mathcal{E})$ is neither true nor false, in the sense that Theorem 5.2 is unsatisfiable: including it makes Sup dishonest for questions requiring the query mechanism, and excluding it makes Sup dishonest for models using the axiom mechanism.*

Proof. Let $\mathcal{E}$ lower to a description logic supporting transitive role axioms whose query language has no closure operator. Then $\Lambda_{\mathcal{E}}$ is faithful for **TRANSITIVE_AXIOM** and unfaithful for **TRANSITIVE_QUERY**. Under Theorem 5.2 with the split alphabet, $\text{Sup}(\mathcal{E})$ contains the first and not the second, and Theorem 5.3 applies. Under the conflated alphabet, honesty demands **TRANSITIVE** $\in \text{Sup}(\mathcal{E})$ (from faithfulness on axioms) and **TRANSITIVE** $\notin \text{Sup}(\mathcal{E})$ (from unfaithfulness on queries) simultaneously. $\square$

Remark 5.8 (Do not make the base relation transitive). A tempting repair for a missing query-side closure is to declare the base role transitive in $\mathcal{T}$. This is not a repair but a change of model: declaring a participation role transitive entails that an entity participates in the participants of its participants, corrupting every other question over the same knowledge base. The correct construction introduces a fresh super-role, asserts *it* transitive, and answers the closure question over the super-role, leaving the base role’s extension untouched.

5.4 Nameability

Proposition 5.9 (An unnameable feature is an undetectable omission). *Let f be a feature of models that is absent from $\mathcal{F}$. Let $\mathcal{E}_1$ lower f faithfully and $\mathcal{E}_2$ ignore it. Then for any question, Theorem 4.2 never reaches a verdict attributing a discrepancy to f , and any differential comparison of $\mathcal{E}_1$ and $\mathcal{E}_2$ compares two different knowledge bases while reporting a difference between two engines.*

Proof. (E1) and (E3) are containment tests over $\mathcal{F}$; a symbol absent from $\mathcal{F}$ occurs in neither $\varrho(q)$ nor $\mathcal{F}(\mathcal{M})$ nor $\text{Sup}(\mathcal{E})$, so neither test can fire on it. The evaluations therefore proceed to (E4)–(E7) in both engines. $\mathcal{E}_2$ ’s program is compiled from $\mathcal{T}$ minus the axioms mentioning f , so its answer sets are those of a knowledge base differing from $\mathcal{E}_1$ ’s. The reported difference is labelled by engine because the engine is the only varying parameter the evaluation records. $\square$

Theorem 5.9 is the formal content of Theorem 2.7: enumerating the constructs one *does* lower is what makes an omission on the other side detectable, and the alphabet must therefore be extended whenever a new construct enters any model, not only when a question requires it.

6 Vacuity

A constraint may survive lowering in form and not in effect. This section formalises the two ways that happens and proves that catching them requires two checks.

Definition 6.1 (Control). Let q carry a *droppable constraint* c —a negation, a cardinality restriction, or a path bound. The *control* q° is q with c removed. The compiled pair is $P = \Lambda_{\mathcal{E}}(\mathcal{M}, q)$, $P^\circ = \Lambda_{\mathcal{E}}(\mathcal{M}, q^\circ)$.

Definition 6.2 (Syntactic vacuity). c is *syntactically vacuous* at $\mathcal{E}$ if $P = P^\circ$ as programs.

Definition 6.3 (Semantic vacuity). c is *semantically vacuous* at $(\mathcal{E}, \mathcal{D})$ if $P \neq P^\circ$ but $A(P) = A(P^\circ)$ on $\mathcal{D}$.

Theorem 6.4 (Irredundance). *Neither check subsumes the other:*

- (i) *there exist $(\mathcal{E}, \mathcal{D}, q, c)$ with c syntactically vacuous and $A(P) = A(P^\circ)$ trivially, which the semantic check reports but with a payload identifying the wrong cause;*
- (ii) *there exist $(\mathcal{E}, \mathcal{D}, q, c)$ with c syntactically present, $P \neq P^\circ$, and $A(P) = A(P^\circ)$, which the syntactic check cannot detect.*

Proof. (ii) is the substantive direction. Let c be a negated conjunction constraining a *role filler*—“ x has no r -successor that is a B ”—and let $\Lambda_{\mathcal{E}}$ construct the negated expression by rooting each negated block at the block’s own first variable. The lowered expression then reads “ x is not itself a B -with-role” rather than “ x has no B -successor”: it is a different, well-formed expression, so $P \neq P^\circ$ and the syntactic check passes. If in addition the model asserts pairwise disjointness between the class of x and the class B , the mis-rooted expression is entailed of every x and the constraint is satisfied identically, giving $A(P) = A(P^\circ)$. Thus a semantic check is required, and its payload—the two identical answer sets—names the actual symptom.

(i) Let $\Lambda_{\mathcal{E}}$ ignore c entirely, so that P and P° are literally the same program. Then $A(P) = A(P^\circ)$ holds trivially and the semantic check fires, but its payload asserts that the constraint made no difference to the answers, when the stronger and more actionable fact is that the constraint never reached the program. A syntactic check reports that fact directly, and reports it without spending the budget of two solver runs. $\square$

Corollary 6.5 (A control must differ from what it controls). *$P \neq P^\circ$ is a precondition for P° to function as a control. A pipeline that constructs the control by textual manipulation of q and recompiles may produce $P^\circ = P$ without any error being raised, in which case the subsequent comparison is between a program and itself and can only report agreement.*

Remark 6.6 (Trigger on droppability, not on a construct name). The set of questions carrying a control must be defined by the property “carries a constraint whose lowering could be silently dropped” rather than by naming one construct such as negation. Cardinality restrictions and path bounds are droppable by the same mechanism—each is a modifier on an otherwise well-formed pattern, so removing it leaves a program that still compiles—and a trigger enumerating only negation admits them silently.

Proposition 6.7 (Vacuity defeats agreement as evidence). *Let $\mathcal{E}_1, \mathcal{E}_2$ agree on q , i.e. $A_1(P_1) = A_2(P_2)$. If c is vacuous at $\mathcal{E}_1$ in either sense, the agreement is not evidence that either lowering is faithful for c .*

Proof. By Theorem 6.3, $A_1(P_1) = A_1(P_1^\circ)$, so $\mathcal{E}_1$ ’s output is the output of the unconstrained question. Agreement with $\mathcal{E}_2$ therefore establishes at most that $\mathcal{E}_2$ agrees with $\mathcal{E}_1$ ’s *unconstrained* answer, which is consistent with $\mathcal{E}_2$ also dropping c and equally consistent with c being extensionally inert on this corpus. No conclusion about faithfulness for c follows. $\square$

7 The blocker trichotomy

We now attach to each unanswered question a statement of what would have to change.

Definition 7.1 (Blocker). For a question q unanswered under $(\mathcal{M}, \mathcal{D}, \mathcal{E}, \tau)$, its *blocker* is:

- blocked-by-model if $\neg \text{Exp}(q, \mathcal{M})$, or if Exp holds but $\neg \text{Der}$ for a reason attributable to a missing axiom (absent closure, absent distinctness);

- blocked-by-corpus if Exp and the axioms suffice but $\mathcal{D}$ contains no assertions populating the question's pattern;
- blocked-by-engine if Exp and Der hold but $\neg \text{Ans}_\tau$.

Definition 7.2 (Unblocking). An *unblocker* for a blocked question is a change δ to exactly one of $\mathcal{M}$, $\mathcal{D}$, $\mathcal{E}$ or τ such that the question is answerable after δ . A blocker record is *actionable* if it names at least one unblocker.

Proposition 7.3 (Blockers are engine-indexed or universally quantified). *A blocked-by-model or blocked-by-corpus record carrying no engine is a claim about all engines. Consequently a measured ANSWERED for the same question and model, by any engine, is a counterexample to that record and not an independent observation compatible with it.*

Proof. By Theorem 7.1, the first two blockers are conditions on $(q, \mathcal{M}, \mathcal{D})$ alone; asserting one without an engine index therefore asserts it for every $\mathcal{E}$. A measured (ANSWERED, $\cdot$) instantiates the negation at one $\mathcal{E}$, falsifying the universal. $\square$

Corollary 7.4 (Superseding, not accumulating). *A record set containing both a universally quantified blocker and a measured ANSWERED for the same question and model asserts a claim and its negation. A system that appends measurements alongside predictions rather than superseding them will hold such a set while every structural well-formedness check passes, because well-formedness checks constrain shape and this defect is in content.*

Proposition 7.5 (A measured claim must agree with its evidence about the model). *Let a record assert (ANSWERED, $\cdot$) for q under model $\mathcal{M}$, citing an experiment e . If e was executed under $\mathcal{M}' \neq \mathcal{M}$, the record is unsupported regardless of the truth of its content.*

Proof. Ans_τ is a four-place predicate including the model. An observation at $\mathcal{M}'$ instantiates the predicate at $\mathcal{M}'$ and is silent at $\mathcal{M}$. $\square$

Remark 7.6 (Which of the two entries is wrong). When Theorem 7.5 fires against a record that contradicts a standing prediction, the natural reading is that the prediction is stale. It may instead be that the prediction is correct and the measured record misattributes its model. Repairing the contradiction by retiring the prediction then deletes the true claim and retains the false one. The check must therefore report which entry failed the agreement test, not merely that a pair disagreed.

Remark 7.7 (Why only the model is compared). A comparison of the engine field is the obvious companion check and is not available when an experiment record carries a scalar engine while differential experiments run two. The asymmetry should be recorded rather than papered over: a check that silently compares the first engine of a pair reports agreement on evidence it did not examine.

8 Divergent semantics as a result

Lowering one model to two target languages does not in general preserve meaning, and the residue is informative.

Proposition 8.1 (Disjointness lowers to a detector). *Let $\mathcal{T}$ assert that concepts A, B are disjoint, and let a rule engine lower this as*

$$\text{viol}(x) \leftarrow A(x), B(x).$$

This is faithful as a detector and unfaithful as a semantics: in a description logic an individual in $A \sqcap B$ renders the knowledge base inconsistent, from which everything follows, while in the rule engine it derives exactly one additional fact.

Proof. The rule is pure positive Datalog and its least model contains $\text{viol}(a)$ exactly for those a in both extensions; no other atom is affected. Under the description-logic reading, $A \sqcap B \sqsubseteq \perp$ together with $A(a), B(a)$ has no model, so every formula is entailed. $\square$

Corollary 8.2 (Localisation). *On a corpus containing a genuine violation, the description-logic engine reports global inconsistency, which identifies that something is wrong without identifying where; the rule engine returns a fact naming the two concepts and the individual. The verdicts differ, both are correct under their respective semantics, and only one is diagnostic.*

Proposition 8.3 (Inherited unique names). *A rule engine over a Herbrand universe satisfies the unique-name assumption unconditionally, since distinct ground terms denote distinct elements. Hence `UNIQUE_NAMES` is not lowered but inherited, and by Theorem 3.7(i) the two engines legitimately disagree on $q_{\geq n}$: the rule engine derives the lower bound the description logic declines to entail.*

Proof. Herbrand interpretations interpret each constant as itself; distinct constants are therefore distinct elements, which is the unique-name assumption. Applying Theorem 3.7(i), the distinctness requirement is met in the rule engine and unmet in the description logic absent explicit assertions. $\square$

Corollary 8.4 (The disagreement is the result). *Under Theorem 8.3 a differential comparison at $q_{\geq n}$ must record disagreement as an expected consequence of the semantics, with the unique-name assumption named as its cause. Recording it as a defect would motivate a repair that could only be effected by falsifying one of the two semantics.*

Remark 8.5 (Inheritance is not vacuity). Theorem 8.3 makes `UNIQUE_NAMES` free, but not unconditionally safe. If the term-construction map is not injective—if two distinct individuals are mapped to the same ground term because both carry the same identifier under the chosen naming scheme—then the inherited assumption silently identifies them, and the lower bound of Theorem 3.7(i) is computed over a collapsed set. A guard checking injectivity of the term map on the individuals occurring in the question is therefore a precondition for the inheritance to be sound.

8.1 Bounding a question can make it cheaper

Proposition 8.6 (Bounded paths are non-recursive). *Let q^∞ ask for reachability under a role r and $q^{\leq k}$ ask for reachability in at most k steps. A lowering of $q^{\leq k}$ by unrolling produces $O(k)$ non-recursive clauses, each depending only on the clause one level below, so evaluation requires no fixpoint. A lowering of q^∞ requires recursion and, on a cyclic corpus, a fixpoint computation.*

Proof. Unrolling defines $r_1 = r$ and $r_{i+1}(x, z) \leftarrow r_i(x, y), r(y, z)$ for $i < k$, with the goal the union of the r_i . The dependency graph on the r_i is a path and hence acyclic, so stratified evaluation in k passes suffices. The unbounded question defines $r^*(x, z) \leftarrow r^*(x, y), r(y, z)$, whose dependency graph has a self-loop. $\square$

Corollary 8.7 (Inversion). *There exist corpora and engines for which $q^{\leq k}$ is answered with certification while q^∞ exceeds the search budget on the same input, and simultaneously a description logic answers q^∞ readily—by a transitive super-role, per Theorem 5.8—while being unable to state $q^{\leq k}$ at all, since a transitive role closes completely and admits no step bound. One word in the question determines which engine can answer it.*

Remark 8.8 (What k inflates). k multiplies the number of clauses, not the size of the search. The unrolled program grows linearly in k while each clause is evaluated over a join whose arity is fixed by the clause shape, so the cost is polynomial in the corpus with the exponent set by that arity rather than by k .

Proposition 8.9 (Recursion shape is a cost, not a capability boundary). *The right-recursive and left-recursive formulations of transitive closure have the same least model and hence the same answers, but their evaluation costs under a goal-directed strategy differ by a factor that grows with the corpus.*

Proof. Both programs define the same relation, so their least models coincide by uniqueness of the least fixpoint of a monotone operator. Under goal-directed evaluation with a fixed selection rule, the left-recursive form re-derives prefixes of every partial path before extending it, while the right-recursive form extends a single frontier; the number of derivation steps therefore differs, with the ratio increasing in the number of paths. $\square$

Remark 8.10 (Assert the shape, not the value). Because the two formulations agree on answers, a test asserting the answer set cannot distinguish them, and a subsequent simplification that replaces one with the other passes every such test while reintroducing the cost. The test must assert the emitted *shape*.

9 One derivation

Definition 9.1 (View). A *view* is a function from a record set to a presentation. Two views v_1, v_2 *disagree* if there is a record set S and a proposition p about the system such that $v_1(S)$ asserts p and $v_2(S)$ asserts $\neg p$.

Theorem 9.2 (Single derivation). *Let d be a derivation function from record sets to derived facts, and let every view factor as $v_i = \pi_i \circ d$ with π_i a pure presentation. Then no two views disagree on any p in the image of d .*

Proof. Suppose $v_1(S)$ asserts p and $v_2(S)$ asserts $\neg p$ with p derived. Then $\pi_1(d(S))$ and $\pi_2(d(S))$ disagree about a component of the single value $d(S)$. Since each π_i presents that component without recomputing it, both read the same value, so they assert the same proposition—contradiction. $\square$

Corollary 9.3 (Duplication admits contradiction). *If two views each compute d independently, as $v_i = \pi_i \circ d_i$ with $d_1 \neq d_2$, then Theorem 9.2 does not apply and disagreement is possible with no schema violation, since both outputs are individually well formed.*

Theorem 9.3 applies to documents as much as to programs. A figure environment duplicated between a manuscript and a caption file is two derivations of one fact, and they drift silently.

Proposition 9.4 (A status field is a claim with no clock). *Let a record carry a status s intended to summarise progress, and let s be set by hand. Then s is a claim about the record’s state at the time of writing, and no subsequent event updates it. In particular a status **planned** on an entity for which completed experiments exist is a false claim that no check over shape can detect.*

Proof. Falsity: **planned** asserts that no run has occurred, and the existence of a completed experiment referencing the entity refutes it. Undetectability by shape: the field is of the correct type and drawn from the permitted enumeration, so schema validation passes. The contradiction is between two records and is therefore a content property. $\square$

Corollary 9.5 (Derive status, or check it against evidence). *Either s is computed by d from the experiment records, in which case Theorem 9.2 applies and it cannot contradict them, or s is authored and a content check must compare it against them. There is no third option in which an authored status remains true by discipline: nobody sets a status backwards to a lie, so the errors are all in the direction of understating progress, and understatement is what makes a coverage table report “nothing has been run” with completed runs on file.*

Remark 9.6 (A gate whose name exceeds its check). A validation gate named for a strong property while testing a weak consequence of it is worse than no gate, because its passing is cited as evidence for the strong property. If a gate is to certify that a derivation used a particular axiom, it must inspect the axiom’s presence; certifying only that the derived count matched an expectation is satisfied by any run whose arithmetic coincides, including runs in which the axiom was absent. For the same reason a count is the weakest of a family of checks: naming the specific entities expected, and specifically naming a *control* entity that must not appear, discriminates where a count cannot.

Remark 9.7 (Hand-copied expectations). An expectation transcribed by hand into a gate is visible in review; an expectation parsed from the artefact under test is only as reliable as the parser, and shares any defect the artefact has. The first is preferable despite being less automatic.

10 Computational validation

10.1 Method

We implement the constructions of Section 2–Section 9 once, in a shared module, and state seventeen propositions as executable experiments. Each experiment reports a verdict in the sense of Theorem 4.1 and a PASS/FAIL against a *pre-registered* expectation recorded in the experiment’s own source before the run. Where a claim is definitional rather than empirical the experiment says so in its **interpretation** field, and where an experiment’s expected outcome is a refusal, the refusal is the PASS condition.

Every experiment that could be satisfied trivially carries a control that is able to fail. We state this requirement precisely because it is the one most easily satisfied in appearance only: a control is a control if and only if it would produce a different result under the negation of the hypothesis, which is exactly Theorem 6.5.

The suite runs on a synthetic knowledge base whose ground truth is known by construction, together with two engines: a description-logic-like evaluator implementing open-world certain-answer semantics without a unique-name assumption, and a rule evaluator implementing least-model semantics over a Herbrand universe. Both are implemented in the shared module so that the experiments test the definitions rather than a third-party implementation’s conformance to them. This is a deliberate limitation and we return to it in Section 11.

10.2 The propositions under test

#	Claim	What the experiment does
01	Theorem 3.4	Constructs the three witnesses and checks each satisfies two predicates and fails the third.
02	Theorem 3.5	Shows five distinct situations produce identical empty answer sets under single-output reporting.
03	Theorem 4.6	Checks no non-ANSWERED verdict carries an answer set, over all questions in the suite.
04	Theorem 3.7	Separates $\geq n$, $\leq n$, $= n$ by which of the two requirements each needs.
05	Theorem 3.8	Contrasts the sound scoped check against the tempting global-boolean check.
06	Theorem 4.10	Scores honest refusal against confident wrong zero under a naive and a verdict-aware rule.
07	Theorem 5.3	Checks the containment test fires exactly when it should, over all question-engine pairs.
08	Theorem 5.4	Attempts the counting lowering, records its rejection, and records the empty certified answer set obtained by removing the gate.
09	Theorem 5.7	Shows the conflated alphabet admits no honest capability set, and the split one does.
10	Theorem 5.9	Runs a differential comparison with a feature absent from the alphabet and confirms the discrepancy is misattributed.
11	Theorem 6.4(ii)	Constructs a mis-rooted negation: syntactically distinct programs, identical answers.
12	Theorem 6.4(i)	Constructs a dropped constraint: identical programs, and confirms the syntactic check reports the stronger fact.
13	Theorem 6.7	Confirms two engines agreeing under a vacuous constraint yields no evidence of faithfulness.
14	Theorem 8.1, Theorem 8.2	Injects a genuine violation; compares global inconsistency against a localising fact.
15	Theorem 8.3, Theorem 8.4	Measures the legitimate disagreement at $q_{\geq n}$ and attributes it.
16	Theorem 8.6, Theorem 8.7	Measures evaluation cost of bounded versus unbounded reachability on a cyclic corpus, both engines.
17	Theorem 9.2, Theorem 9.4	Builds two views over one derivation and two over duplicated derivations; injects a stale status and checks which regime detects it.

Table 1: The seventeen validation experiments and the claims they test. Experiments 08, 11, 12 and 15 are ones whose PASS condition is a refusal or a disagreement rather than an answer.

10.3 Results

Results are written as one JSON document per experiment together with a summary document, in `ckg/validation-incompleteness/results/`. Each document records the pre-registered expectation, the measured outcome, the verdict, the control’s outcome, and an interpretation stating what the result does *not* establish. We draw attention to four outcomes.

Experiment 06. Under a scoring rule counting answers returned, the engine giving a confident wrong zero outcores the honest refuser; under a verdict-aware rule the ordering reverses. The magnitude of the reversal is the ground-truth cardinality, as Theorem 4.10 predicts.

Experiment 08. The counting lowering is rejected by the fragment gate. With the gate removed the run terminates and returns an empty answer set with certification asserted—the answer set of a program whose aggregate never grounded. This is the configuration Theorem 4.7 identifies as maximally misleading, and it is reachable by removing a check that an audit conducted against the target language would recommend removing.

Experiment 11. The mis-rooted negation produces programs that differ syntactically and answer identically. The syntactic check passes and the semantic check fires, confirming Theorem 6.4(ii): a syntactic guard cannot catch a semantic vacuity.

Experiment 16. Bounded reachability is answered with certification where unbounded reachability on the same cyclic corpus exceeds the search budget, while the open-world engine answers the unbounded question readily and returns NO-QUERY-SURFACE on the bounded one. Both directions of Theorem 8.7 are realised on a single corpus, at one budget, on twelve assertions. The refusal is NO-QUERY-SURFACE rather than CANNOT-EXPRESS because the model does carry BOUNDED_PATH: the limit is in the engine’s compiler, not in the language, and it is exactly the distinction of Theorem 3.4 that lets the two be told apart. Measured step counts for $k = 1, \dots, 5$ are 1, 2, 3, 4, 5, confirming the cost claim of Theorem 8.6 that unrolling scales with the bound and not with the corpus.

10.4 What the suite does not establish

The engines are ours, so conformance of any third-party reasoner to Theorem 3.2 is not tested; the corpus is synthetic, so no claim about biochemical coverage follows; and the timing in Experiment 16 is a property of our evaluator’s strategy, not of any standard. Experiments 03, 07 and 09 are definitional—they check that an implementation obeys a definition—and cannot fail except by implementation error, which is what they are for.

11 Discussion

11.1 What the framework buys

The framework converts a class of silent defects into loud ones. Each of the five situations in §1.1 now produces a distinct, structured output, and Theorem 4.6 makes the distinctness a checkable property rather than a convention. The two vacuity checks convert a defect that previously manifested as a correct-looking answer into a verdict, and Theorem 6.4 shows that implementing only one of them leaves a gap. Theorem 9.2 converts a class of documentation drift into an impossibility.

The cost is real. A verdict-returning system is more work to build than one returning answer sets, its outputs are harder to aggregate—one cannot average a verdict—and its refusals will be read by some consumers as failures. The last is not a presentational problem but a scoring one, and Theorem 4.10 identifies the scoring rules under which it occurs.

11.2 What the framework does not do

It does not make a knowledge graph complete, and it is not intended to. It does not tell you which of the three blockers is cheapest to remove; the unblocking relation of Theorem 7.2 names candidates and says nothing about their cost. It does not detect a lowering that is unfaithful in a way that changes answers without making a constraint inert—a mis-lowering that yields *different wrong* answers passes both vacuity checks, and only a comparison against ground truth or against an independent engine catches it, with Theorem 6.7 limiting what such a comparison establishes.

Most importantly, the framework does not remove the need for judgement about which questions to ask. A question set assembled to demonstrate a system’s strengths will produce a

verdict distribution favourable to that system, and Theorem 4.6 makes the individual verdicts honest without making the aggregate representative. A fixture built to exhibit a property is the weakest available evidence that anything else has it.

11.3 Outlook

Two extensions seem worth pursuing. The first is a cost model over the unblocking relation, turning Theorem 7.2 from an enumeration into a ranking: given a blocked question set, which single change to model, corpus, engine or budget unblocks the most. The second is the treatment of *partial* derivability—questions determined on part of their domain and underdetermined elsewhere—which the present binary Der predicate collapses, and which is the common case on real corpora where closure holds for curated entities and fails for imported ones.

Reproducibility

The validation suite is deterministic; every stochastic component seeds explicitly. Each result document records the expectation before the measured value, and expectations are transcribed by hand into the experiment sources rather than parsed from the artefacts under test, for the reason given in Theorem 9.7. Result documents record `null` where nothing was measured, and never a zero.

References

- Rafael Alcántara, Kristian B. Axelsen, Anne Morgat, Eugeni Belda, Elisabeth Coudert, Alan Bridge, Hong Cao, Paula de Matos, Marcus Ennis, Steve Turner, Gareth Owen, Lydie Bouguéret, Ioannis Xenarios, and Christoph Steinbeck. Rhea—a manually curated resource of biochemical reactions. *Nucleic Acids Research*, 40(D1):D754–D760, 2012. doi: 10.1093/nar/gkr1126.
- Franz Baader, Ian Horrocks, Carsten Lutz, and Uli Sattler. *An Introduction to Description Logic*. Cambridge University Press, Cambridge, 2017. doi: 10.1017/9781139025355.
- Parit Bansal, Anne Morgat, Kristian B. Axelsen, Venkatesh Muthukrishnan, Elisabeth Coudert, Lucila Aimó, Nevila Hyka-Nouspikel, Elisabeth Gasteiger, Arnaud Kerhornou, Teresa Batista Neto, Monica Pozzato, Marie-Claude Blatter, Alex Ignatchenko, Nicole Redaschi, and Alan Bridge. Rhea, the reaction knowledgebase in 2022. *Nucleic Acids Research*, 50(D1):D693–D700, 2022. doi: 10.1093/nar/gkab1016.
- Ilan Beer, Shoham Ben-David, Cindy Eisner, and Yoav Rodeh. Efficient detection of vacuity in temporal model checking. *Formal Methods in System Design*, 18(2):141–163, 2001. doi: 10.1023/A:1008779610539.
- Kirill Degtyarenko, Paula de Matos, Marcus Ennis, Janna Hastings, Martin Zbinden, Alan McNaught, Rafael Alcántara, Michael Darsow, Mickaël Guedj, and Michael Ashburner. ChEBI: a database and ontology for chemical entities of biological interest. *Nucleic Acids Research*, 36(suppl_1):D344–D350, 2008. doi: 10.1093/nar/gkm791.
- Francesco M. Donini. Complexity of reasoning. *The Description Logic Handbook: Theory, Implementation and Applications*, pages 96–136, 2003. Cambridge University Press.
- Birte Glimm, Ian Horrocks, Boris Motik, Giorgos Stoilos, and Zhe Wang. HermiT: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014. doi: 10.1007/s10817-014-9305-1.
- Bernardo Cuenca Grau, Ian Horrocks, Boris Motik, Bijan Parsia, Peter Patel-Schneider, and Ulrike Sattler. OWL 2: The next step for OWL. *Journal of Web Semantics*, 6(4):309–322, 2008. doi: 10.1016/j.websem.2008.05.001.

- Yuanbo Guo, Zhengxiang Pan, and Jeff Heflin. LUBM: A benchmark for OWL knowledge base systems. *Journal of Web Semantics*, 3(2–3):158–182, 2005. doi: 10.1016/j.websem.2005.06.005.
- Janna Hastings, Gareth Owen, Adriano Dekker, Marcus Ennis, Namrata Kale, Venkatesh Muthukrishnan, Steve Turner, Neil Swainston, Pedro Mendes, and Christoph Steinbeck. ChEBI in 2016: Improved services and an expanding collection of metabolites. *Nucleic Acids Research*, 44(D1):D1214–D1219, 2016. doi: 10.1093/nar/gkv1031.
- Ian Horrocks, Peter F. Patel-Schneider, and Frank van Harmelen. From SHIQ and RDF to OWL: the making of a web ontology language. *Journal of Web Semantics*, 1(1):7–26, 2003. doi: 10.1016/j.websem.2003.07.001.
- Yevgeny Kazakov, Markus Krötzsch, and František Simčík. The incredible ELK: From polynomial procedures to efficient reasoning with $\mathcal{EL}$ ontologies. *Journal of Automated Reasoning*, 53(1):1–61, 2014. doi: 10.1007/s10817-013-9296-3.
- Orna Kupferman and Moshe Y. Vardi. Vacuity detection in temporal model checking. *International Journal on Software Tools for Technology Transfer*, 4(2):224–233, 2003. doi: 10.1007/s100090100062.
- Boris Motik, Bernardo Cuenca Grau, Ian Horrocks, Zhe Wu, Achille Fokoue, and Carsten Lutz. OWL 2 web ontology language profiles. W3C recommendation, World Wide Web Consortium, 2009a. URL <https://www.w3.org/TR/owl2-profiles/>.
- Boris Motik, Rob Shearer, and Ian Horrocks. Hypertableau reasoning for description logics. *Journal of Artificial Intelligence Research*, 36:165–228, 2009b. doi: 10.1613/jair.2811.
- Bijan Parsia, Nicolas Matentzoglou, Rafael S. Gonçalves, Birte Glimm, and Andreas Steigmiller. The OWL reasoner evaluation (ORE) 2015 competition report. *Journal of Automated Reasoning*, 59(4):455–482, 2017. doi: 10.1007/s10817-017-9406-8.
- Gunjan Singh, Sumit Bhatia, and Raghava Mutharaju. OWL2Bench: A benchmark for OWL 2 reasoners. *Lecture Notes in Computer Science*, 12507:81–96, 2020. doi: 10.1007/978-3-030-62466-8_6.